
name: vibe-coding-fullstack-dev description: > Apply this skill whenever the task involves writing code, building web applications, developing software, designing interfaces, creating animations, scaffolding projects, debugging, refactoring, architecting systems, or any context where something needs to be built. Triggers include: "build me", "create", "code this", "make an app", "fix this bug", "design a UI", "animate this", "set up a project", "write a component", "deploy this", "optimize this", "review my code", or any message containing code or a technical specification. This skill governs how the agent thinks, builds, designs, and ships — at the intersection of engineering precision and creative velocity. Read this before producing any code, design, or technical recommendation — your defaults alone do not reach this bar.

This agent builds things that work, look exceptional, and ship fast. It holds two capabilities in constant tension: the engineering discipline to build things correctly, and the creative velocity to build things quickly. It knows which mode the moment demands and switches without hesitation.

The standard is not "functional." The standard is: **would a senior engineer and a lead designer both look at this and feel proud?**

The Builder's Philosophy

Velocity is a feature. A working prototype today is worth more than a perfect system next month. Ship something. Learn from it. Improve it.

But velocity without craft creates debt. Moving fast without thinking creates systems that slow you down permanently. The fastest path to done is the one you don't have to redo.

Design and code are the same decision. A button's padding is a visual decision and a code decision simultaneously. The best builders don't hand off between "design" and "dev" — they hold both.

The user doesn't care how the code is structured. But the user absolutely cares if the interface feels wrong, loads slowly, or breaks unexpectedly. Build for the person using it.

Read the room, then build for it. A startup MVP demands different decisions than a production enterprise system. A personal portfolio demands different decisions than a SaaS dashboard. Context determines craft level — but craft is never zero.

Part 1 — Vibe Coding

1.1 — What Vibe Coding Actually Is

Vibe coding is not sloppy coding. It is **high-velocity creative development** — the practice of moving from idea to working product as fast as possible, using intuition, momentum, and intelligent shortcuts without sacrificing the foundations that matter.

THE VIBE CODING MANIFESTO

DONE > PERFECT (initially)

A running app teaches you more than a perfect plan.

Ship something you can touch and improve.

SCAFFOLD FAST, BUILD CAREFULLY

Use every scaffold, generator, and template available.

They eliminate setup so you can focus on what's unique.

Understand what they generated — don't treat it as magic.

FEEL THE FLOW

When the code is writing itself, don't stop to over-engineer.

Ride the momentum. Clean up when momentum naturally pauses.

SHIP → LEARN → ITERATE

The real spec emerges from using the thing.

Version 1 is always wrong. Ship it so you can write version 2.

INSTINCT + VERIFICATION

Trust your instincts on architecture and design.

Verify with tests, type safety, and a working demo.

Never trust instincts on security or data integrity — verify those always.

1.2 — The Vibe Coding Toolkit

Scaffolding tools — know these cold:

```
# React / Next.js
npx create-next-app@latest my-app --typescript --tailwind --eslint --app
npx create-react-app my-app --template typescript

# Vite (faster, leaner)
npm create vite@latest my-app -- --template react-ts
npm create vite@latest my-app -- --template vue-ts
npm create vite@latest my-app -- --template svelte-ts

# Full-stack frameworks
npx create-t3-app@latest           # T3 stack: Next.js + tRPC + Prisma + Tailwind
npm create astro@latest           # Astro: content-first sites
npx nuxt@latest init my-app       # Nuxt 3
cargo new my-project              # Rust
go mod init github.com/user/proj  # Go module
python -m vite my-project         # Python with Vite frontend
django-admin startproject mysite  # Django

# Package managers (always use one consistently)
pnpm install    # preferred: fast, disk-efficient
yarn install    # good alternative
npm install     # universal fallback
bun install     # fastest, growing ecosystem
```

Component libraries for speed — pick one per project:

shadcn/ui	— composable, unstyled-base, Tailwind, copy-paste ownership
Radix UI	— headless primitives, full ally, no styling
Headless UI	— Tailwind team, accessible components
Mantine	— batteries included, excellent DX, full component set
Chakra UI	— accessible, themeable, React
Ant Design	— enterprise-grade, massive component set
MUI (Material UI)	— Google Material Design, most popular
NextUI	— beautiful defaults, modern aesthetic
Tremor	— dashboard components, analytics UI
daisyUI	— Tailwind plugin, CSS classes, any framework

Backend quick-start:

Hono	— ultra-fast, lightweight, edge-ready, any runtime
Fastify	— Node.js, fastest JSON HTTP framework

Express	– universal, massive ecosystem, battle-tested
NestJS	– structured, enterprise, decorators, opinionated
FastAPI	– Python, async, auto-docs, type hints
Django REST	– Python, batteries included, admin panel free
Elysia	– Bun-native, type-safe, blazing fast
Remix	– full-stack React, web standards, progressive enhancement

Database quick-start:

Prisma	– TypeScript ORM, excellent DX, auto-migration
Drizzle	– lightweight TypeScript ORM, SQL-first, fast
SQLModel	– Python, Pydantic + SQLAlchemy, FastAPI native
Mongoose	– MongoDB + Node.js, schema with flexibility
Supabase	– PostgreSQL as a service, auth, storage, realtime
PlanetScale	– MySQL serverless, branching, excellent DX
Neon	– PostgreSQL serverless, branching, fast cold start
Turso	– SQLite at the edge, libSQL, global distribution

Auth in minutes:

Clerk	– best DX, full UI, every provider, webhooks
Auth.js / NextAuth	– open source, any provider, framework flexible
Supabase Auth	– built-in with Supabase, RLS integration
Lucia	– lightweight, full control, TypeScript
Better Auth	– modern, type-safe, growing fast

Deployment — from zero to live:

Vercel	– Next.js native, best DX, edge functions, analytics
Netlify	– framework-agnostic, forms, edge, good free tier
Railway	– any app/database, simple config, great DX
Fly.io	– Docker-based, global, persistent storage, excellent for APIs
Render	– simple, free tier, cron jobs, background workers
Cloudflare Pages	– edge-first, Workers, D1, R2, KV – very fast
Coolify	– self-hosted PaaS, deploy anything, your server

1.3 — The Vibe Build Sequence

PHASE 1 — CONCEPT (5 minutes)

What is the ONE thing this must do?

Who uses it?

What does "working" look like?

Answer these before touching the keyboard.

PHASE 2 — SCAFFOLD (5-15 minutes)

Pick the right stack for the job (not the newest stack)

Run the scaffolding command

Install core dependencies

Set up environment variables

Verify: does it run?

PHASE 3 — SKELETON (30-60 minutes)

Build the basic structure — routing, layout, main components

No data, no backend, no styling yet

Get the bones right before adding muscle

Verify: can you navigate the entire app?

PHASE 4 — CORE FUNCTIONALITY (hours to days)

Build the thing the app actually does

Start with the happy path

No error handling yet — that comes next

Verify: does the main use case work end-to-end?

PHASE 5 — HARDEN (hours)

Add error handling at every failure point

Add loading states

Handle edge cases you know will happen

Add form validation

Verify: does it survive bad input and slow connections?

PHASE 6 — POLISH (hours)

Styling and animations

Responsive behavior

Accessibility basics

Performance optimization

Verify: would you be comfortable showing this to someone?

PHASE 7 — SHIP

Deploy

Set up monitoring

Document the essentials

Then: iterate based on what you learn

Part 2 — Frontend Engineering

2.1 — React Mastery

Hooks — know these deeply:

```

// useState - local component state
const [count, setCount] = useState<number>(0)
const [user, setUser] = useState<User | null>(null)

// useEffect - side effects, subscriptions, cleanup
useEffect(() => {
  const subscription = subscribe(id)
  return () => subscription.unsubscribe() // cleanup is critical
}, [id]) // dependency array - empty = once, [dep] = on dep change

// useCallback - memoize functions (prevents unnecessary re-renders)
const handleClick = useCallback((id: string) => {
  dispatch({ type: 'SELECT', payload: id })
}, [dispatch]) // only recreate when dispatch changes

// useMemo - memoize expensive computations
const sortedItems = useMemo(() =>
  items.sort((a, b) => a.name.localeCompare(b.name)),
  [items]
)

// useRef - persistent mutable value, DOM access
const inputRef = useRef<HTMLInputElement>(null)
const timerRef = useRef<NodeJS.Timeout>()
// Access DOM: inputRef.current?.focus()

// useContext - consume context without prop drilling
const { user, theme } = useContext(AppContext)

// useReducer - complex state logic, replaces multiple useState
const [state, dispatch] = useReducer(reducer, initialState)
dispatch({ type: 'INCREMENT', payload: 1 })

// Custom hooks - the real power
function useDebounce<T>(value: T, delay: number): T {
  const [debouncedValue, setDebouncedValue] = useState(value)
  useEffect(() => {
    const handler = setTimeout(() => setDebouncedValue(value), delay)
    return () => clearTimeout(handler)
  }, [value, delay])
  return debouncedValue
}

```

Component patterns:

```

// Compound Component pattern

```

```

const Select = ({ children, value, onChange }: SelectProps) => {
  return (
    <SelectContext.Provider value={{ value, onChange }}>
      <div className="select-wrapper">{children}</div>
    </SelectContext.Provider>
  )
}
Select.Option = SelectOption
Select.Trigger = SelectTrigger
// Usage: <Select><Select.Trigger /><Select.Option /></Select>

// Render Props pattern
const DataFetcher = ({ url, render }: {
  url: string
  render: (data: Data, loading: boolean) => ReactNode
}) => {
  const { data, loading } = useFetch(url)
  return render(data, loading)
}

// Forward Ref pattern (for exposing DOM nodes)
const Input = forwardRef<HTMLInputElement, InputProps>(
  ({ className, ...props }, ref) => (
    <input ref={ref} className={cn("base-input", className)} {...props} />
  )
)
Input.displayName = "Input"

// Polymorphic component (as prop)
type ButtonProps<T extends ElementType> = {
  as?: T
  children: ReactNode
} & ComponentPropsWithoutRef<T>

const Button = <T extends ElementType = 'button'>({
  as, children, ...props
}: ButtonProps<T>) => {
  const Component = as ?? 'button'
  return <Component {...props}>{children}</Component>
}
// <Button as="a" href="/home">Link Button</Button>

```

Performance patterns:

```

// Code splitting with React.lazy
const HeavyComponent = lazy(() => import('./HeavyComponent'))

```

```

// Wrap in Suspense:
<Suspense fallback={<Skeleton />}>
  <HeavyComponent />
</Suspense>

// Virtualization for long lists
import { useVirtualizer } from '@tanstack/react-virtual'
// Render only visible rows – critical for 1000+ item lists

// React.memo – skip re-render if props unchanged
const ExpensiveChild = memo(({ data }: { data: Data }) => (
  <div>{/* expensive render */}</div>
))

// Key prop discipline – never use array index as key for dynamic lists
// Good:
items.map(item => <Item key={item.id} {...item} />)
// Bad (causes state bugs):
items.map((item, index) => <Item key={index} {...item} />)

```

2.2 — Next.js App Router Architecture

```

// App Router folder structure
app/
  layout.tsx           // Root layout (persistent shell)
  page.tsx             // Route component
  loading.tsx          // Automatic loading UI (Suspense boundary)
  error.tsx            // Error boundary for this route
  not-found.tsx        // 404 for this segment
  (auth)/              // Route group (no URL segment)
    login/page.tsx
    register/page.tsx
  dashboard/
    layout.tsx         // Nested layout
    page.tsx
    [id]/              // Dynamic segment
      page.tsx
  api/
    users/route.ts     // API route handler

// Server Component (default in App Router)
// Runs on server. Can await. Cannot use hooks or browser APIs.
async function UserProfile({ id }: { id: string }) {
  const user = await db.user.findUnique({ where: { id } })

```



```

    if (!user) notFound()
    return <div>{user.name}</div>
  }

// Client Component - add "use client" directive
"use client"
import { useState } from 'react'
function Counter() {
  const [count, setCount] = useState(0)
  return <button onClick={() => setCount(c => c + 1)}>{count}</button>
}

// Route Handler (API routes in App Router)
// app/api/users/route.ts
import { NextRequest, NextResponse } from 'next/server'
export async function GET(req: NextRequest) {
  const users = await db.user.findMany()
  return NextResponse.json(users)
}
export async function POST(req: NextRequest) {
  const body = await req.json()
  const user = await db.user.create({ data: body })
  return NextResponse.json(user, { status: 201 })
}

// Server Actions - forms without API routes
async function createUser(formData: FormData) {
  "use server"
  const name = formData.get('name') as string
  await db.user.create({ data: { name } })
  revalidatePath('/users')
}

// Data fetching patterns
// Static (cached, fastest):
const data = await fetch('https://api.example.com/data')

// Dynamic (no cache):
const data = await fetch('https://api.example.com/data', {
  cache: 'no-store'
})

// Revalidated (ISR):
const data = await fetch('https://api.example.com/data', {
  next: { revalidate: 3600 } // seconds
})

```

```
// Metadata
export const metadata: Metadata = {
  title: 'Page Title',
  description: 'Page description',
  openGraph: { title: 'OG Title', images: ['/og-image.png'] }
}
```

2.3 — TypeScript — Always, Correctly

```
// Core type patterns every professional uses

// Utility types
type Partial<T>           // All properties optional
type Required<T>         // All properties required
type Readonly<T>         // All properties readonly
type Pick<T, K>           // Select subset of properties
type Omit<T, K>           // Remove subset of properties
type Exclude<T, U>        // Exclude types from union
type Extract<T, U>        // Extract types from union
type NonNullable<T>      // Remove null and undefined
type ReturnType<T>       // Get function return type
type Parameters<T>       // Get function parameters as tuple
type Awaited<T>          // Unwrap Promise type

// Discriminated unions — the most underused pattern
type Result<T, E = Error> =
  | { success: true; data: T }
  | { success: false; error: E }

function parseUser(input: unknown): Result<User> {
  if (!isUser(input)) {
    return { success: false, error: new Error('Invalid user') }
  }
  return { success: true, data: input }
}

// Generic constraints
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key]
}

// Template literal types
type EventName = `on${Capitalize<string>}`
type CSSProperty = `${string}-${string}`
```

```
// Conditional types
type IsArray<T> = T extends any[] ? true : false
type Flatten<T> = T extends Array<infer U> ? U : T

// Zod for runtime validation + TypeScript types
import { z } from 'zod'
const UserSchema = z.object({
  id: z.string().uuid(),
  email: z.string().email(),
  age: z.number().min(0).max(150),
  role: z.enum(['admin', 'user', 'moderator']),
  createdAt: z.date().optional()
})
type User = z.infer<typeof UserSchema> // TypeScript type from schema
const user = UserSchema.parse(rawData) // Runtime validation

// Strict function signatures
// Bad:
function process(data: any): any { }
// Good:
function process<T extends Record<string, unknown>>(
  data: T,
  options?: ProcessOptions
): ProcessResult<T> { }
```

2.4 — State Management

```
// ZUSTAND — lightweight, no boilerplate, recommended for most apps
import { create } from 'zustand'
import { persist, devtools } from 'zustand/middleware'

interface AppStore {
  user: User | null
  setUser: (user: User | null) => void
  theme: 'light' | 'dark'
  toggleTheme: () => void
}

const useAppStore = create<AppStore>() (
  devtools(
    persist(
      (set) => ({
        user: null,
```

```

        setUser: (user) => set({ user }),
        theme: 'light',
        toggleTheme: () => set((state) => ({
            theme: state.theme === 'light' ? 'dark' : 'light'
        })))
    }},
    { name: 'app-store' }
)
)
)

// REACT QUERY / TANSTACK QUERY – server state, the right tool
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'

function useUsers() {
    return useQuery({
        queryKey: ['users'],
        queryFn: () => fetch('/api/users').then(r => r.json()),
        staleTime: 5 * 60 * 1000, // 5 minutes
    })
}

function useCreateUser() {
    const queryClient = useQueryClient()
    return useMutation({
        mutationFn: (data: CreateUserInput) =>
            fetch('/api/users', {
                method: 'POST',
                body: JSON.stringify(data)
            }).then(r => r.json()),
        onSuccess: () => {
            queryClient.invalidateQueries({ queryKey: ['users'] })
        }
    })
}

// JOTAI – atomic state, excellent for complex interdependencies
import { atom, useAtom, useAtomValue, useSetAtom } from 'jotai'
const countAtom = atom(0)
const doubleAtom = atom((get) => get(countAtom) * 2) // derived atom

// WHEN TO USE WHAT
// Local UI state (toggle, form field) → useState
// Complex local state (form, multi-step) → useReducer
// Server data (fetching, caching, sync) → TanStack Query
// Global client state (user, theme, cart) → Zustand or Jotai
// URL state (filters, pagination) → URL search params + useSearchParams

```

Part 3 — UI/UX Design in Code

3.1 — Design Systems in Tailwind

```
// tailwind.config.ts — your design system lives here
import type { Config } from 'tailwindcss'

const config: Config = {
  darkMode: ['class'],
  content: ['./src/**/*.{ts,tsx}'],
  theme: {
    extend: {
      colors: {
        // Semantic color tokens — not descriptive names
        background: {
          DEFAULT: 'hsl(var(--background))',
          secondary: 'hsl(var(--background-secondary))',
        },
        foreground: {
          DEFAULT: 'hsl(var(--foreground))',
          muted: 'hsl(var(--foreground-muted))',
        },
        primary: {
          DEFAULT: 'hsl(var(--primary))',
          foreground: 'hsl(var(--primary-foreground))',
          hover: 'hsl(var(--primary-hover))',
        },
        border: 'hsl(var(--border))',
        ring: 'hsl(var(--ring))',
      },
      spacing: {
        // Extend with project-specific spacing if needed
        '18': '4.5rem',
        '88': '22rem',
      },
      fontFamily: {
        sans: ['var(--font-sans)', 'system-ui', 'sans-serif'],
        mono: ['var(--font-mono)', 'monospace'],
      },
      fontSize: {
        // Semantic type scale
        'display': ['3.5rem', { lineHeight: '1.1', letterSpacing: '-0.02em' }],
        'headline-lg': ['2.25rem', { lineHeight: '1.2', letterSpacing: '-0.01em' }],
      },
    },
  },
}
```

```

    'headline-md': ['1.5rem', { lineHeight: '1.3' }],
    'headline-sm': ['1.25rem', { lineHeight: '1.4' }],
    'body-lg': ['1.125rem', { lineHeight: '1.6' }],
    'body-md': ['1rem', { lineHeight: '1.6' }],
    'body-sm': ['0.875rem', { lineHeight: '1.5' }],
    'label': ['0.75rem', { lineHeight: '1.4', letterSpacing: '0.05em' }],
  },
  borderRadius: {
    // Consistent radius system
    'sm': '0.25rem',
    'md': '0.5rem',
    'lg': '0.75rem',
    'xl': '1rem',
    '2xl': '1.5rem',
  },
  boxShadow: {
    'elevation-1': '0 1px 3px rgba(0,0,0,0.06), 0 1px 2px rgba(0,0,0,0.04)',
    'elevation-2': '0 4px 6px rgba(0,0,0,0.07), 0 2px 4px rgba(0,0,0,0.04)',
    'elevation-3': '0 10px 15px rgba(0,0,0,0.08), 0 4px 6px rgba(0,0,0,0.04)',
    'elevation-4': '0 20px 25px rgba(0,0,0,0.1), 0 10px 10px rgba(0,0,0,0.04)',
  },
  keyframes: {
    'fade-in': {
      from: { opacity: '0' },
      to: { opacity: '1' },
    },
    'slide-up': {
      from: { opacity: '0', transform: 'translateY(10px)' },
      to: { opacity: '1', transform: 'translateY(0)' },
    },
    'slide-down': {
      from: { opacity: '0', transform: 'translateY(-10px)' },
      to: { opacity: '1', transform: 'translateY(0)' },
    },
    'scale-in': {
      from: { opacity: '0', transform: 'scale(0.95)' },
      to: { opacity: '1', transform: 'scale(1)' },
    },
  },
  animation: {
    'fade-in': 'fade-in 0.2s ease-out',
    'slide-up': 'slide-up 0.3s ease-out',
    'slide-down': 'slide-down 0.3s ease-out',
    'scale-in': 'scale-in 0.2s ease-out',
  },
},
},

```

```

    plugins: [
      require('tailwindcss-animate'),
      require('@tailwindcss/typography'),
      require('@tailwindcss/forms'),
    ],
  },
}

export default config

```

CSS custom properties for theming:

```

/* globals.css */
:root {
  --background: 0 0% 100%;
  --background-secondary: 0 0% 97%;
  --foreground: 222.2 84% 4.9%;
  --foreground-muted: 215.4 16.3% 46.9%;
  --primary: 222.2 47.4% 11.2%;
  --primary-foreground: 210 40% 98%;
  --primary-hover: 222.2 47.4% 20%;
  --border: 214.3 31.8% 91.4%;
  --ring: 222.2 84% 4.9%;
  --radius: 0.5rem;
}

.dark {
  --background: 222.2 84% 4.9%;
  --background-secondary: 217.2 32.6% 12%;
  --foreground: 210 40% 98%;
  --foreground-muted: 215 20.2% 65.1%;
  --primary: 210 40% 98%;
  --primary-foreground: 222.2 47.4% 11.2%;
  --primary-hover: 210 40% 90%;
  --border: 217.2 32.6% 17.5%;
  --ring: 212.7 26.8% 83.9%;
}

```

3.2 — Component Architecture

The `cn()` utility — always use it:

```

// lib/utils.ts
import { type ClassValue, clsx } from 'clsx'
import { twMerge } from 'tailwind-merge'

```

```

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
// Why: clsx handles conditional classes, twMerge resolves Tailwind conflicts
// cn("px-4 px-6") → "px-6" (correct, not "px-4 px-6" which is ambiguous)

```

Building a proper Button component:

```

// components/ui/button.tsx
import { cva, type VariantProps } from 'class-variance-authority'
import { forwardRef } from 'react'
import { cn } from '@lib/utils'

const buttonVariants = cva(
  // Base styles – applied to all variants
  'inline-flex items-center justify-center rounded-md text-sm font-medium ' +
  'ring-offset-background transition-colors focus-visible:outline-none ' +
  'focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 ' +
  'disabled:pointer-events-none disabled:opacity-50',
  {
    variants: {
      variant: {
        default: 'bg-primary text-primary-foreground hover:bg-primary/90',
        secondary: 'bg-secondary text-secondary-foreground hover:bg-secondary/80',
        outline: 'border border-input bg-background hover:bg-accent hover:text-accent-foreground',
        ghost: 'hover:bg-accent hover:text-accent-foreground',
        link: 'text-primary underline-offset-4 hover:underline',
        destructive: 'bg-destructive text-destructive-foreground hover:bg-destructive',
      },
      size: {
        sm: 'h-9 rounded-md px-3',
        md: 'h-10 px-4 py-2',
        lg: 'h-11 rounded-md px-8',
        icon: 'h-10 w-10',
      },
    },
    defaultVariants: {
      variant: 'default',
      size: 'md',
    },
  }
)

interface ButtonProps
  extends React.ButtonHTMLAttributes<HTMLButtonElement>,
    VariantProps<typeof buttonVariants> {

```



```

    isLoading?: boolean
  }

const Button = forwardRef<HTMLButtonElement, ButtonProps>(
  ({ className, variant, size, isLoading, children, disabled, ...props }, ref) =>
    <button
      ref={ref}
      className={cn(buttonVariants({ variant, size }), className)}
      disabled={disabled || isLoading}
      {...props}
    >
      {isLoading && <Spinner className="mr-2 h-4 w-4 animate-spin" />}
      {children}
    </button>
  )
)

Button.displayName = 'Button'
export { Button, buttonVariants }

```

3.3 — Animations

Framer Motion — the professional standard:

```

import { motion, AnimatePresence, useScroll, useTransform,
  useMotionValue, useSpring, useInView } from 'framer-motion'
import { useRef } from 'react'

// Basic animation
<motion.div
  initial={{ opacity: 0, y: 20 }}
  animate={{ opacity: 1, y: 0 }}
  exit={{ opacity: 0, y: -20 }}
  transition={{ duration: 0.3, ease: 'easeOut' }}
/>

// AnimatePresence — for mount/unmount animations
<AnimatePresence mode="wait">
  {isOpen && (
    <motion.div
      key="modal"
      initial={{ opacity: 0, scale: 0.95 }}
      animate={{ opacity: 1, scale: 1 }}
      exit={{ opacity: 0, scale: 0.95 }}
      transition={{ duration: 0.2 }}
    />
  )}

```

```

    })
  </AnimatePresence>

  // Stagger children animations
  const containerVariants = {
    hidden: { opacity: 0 },
    visible: {
      opacity: 1,
      transition: {
        staggerChildren: 0.08, // delay between each child
        delayChildren: 0.1,
      }
    }
  }

  const itemVariants = {
    hidden: { opacity: 0, y: 20 },
    visible: { opacity: 1, y: 0, transition: { ease: 'easeOut' } }
  }

  <motion.ul variants={containerVariants} initial="hidden" animate="visible">
    {items.map(item => (
      <motion.li key={item.id} variants={itemVariants}>{item.name}</motion.li>
    ))}
  </motion.ul>

  // Scroll-triggered animations
  function ScrollReveal({ children }: { children: React.ReactNode }) {
    const ref = useRef(null)
    const isInView = useInView(ref, { once: true, margin: "-100px" })
    return (
      <motion.div
        ref={ref}
        initial={{ opacity: 0, y: 40 }}
        animate={isInView ? { opacity: 1, y: 0 } : {}}
        transition={{ duration: 0.6, ease: [0.25, 0.1, 0.25, 1] }}
      >
        {children}
      </motion.div>
    )
  }

  // Parallax scroll effect
  function ParallaxSection() {
    const { scrollYProgress } = useScroll()
    const y = useTransform(scrollYProgress, [0, 1], ['0%', '50%'])
    return <motion.div style={{ y }}>{/* content */}</motion.div>
  }

```

```

// Spring-based gesture animations
function DraggableCard() {
  const x = useMotionValue(0)
  const y = useMotionValue(0)
  const rotateX = useTransform(y, [-100, 100], [15, -15])
  const rotateY = useTransform(x, [-100, 100], [-15, 15])

  return (
    <motion.div
      style={{ x, y, rotateX, rotateY }}
      drag
      dragConstraints={{ left: 0, right: 0, top: 0, bottom: 0 }}
      dragElastic={0.1}
      whileHover={{ scale: 1.02 }}
      whileTap={{ scale: 0.98 }}
    />
  )
}

// Layout animations (FLIP technique, automatic)
<motion.div layout layoutId="card-1">
  /* Automatically animates layout changes */
</motion.div>

// Shared element transitions
// Element 1:
<motion.img layoutId="hero-image" src={img} />
// Element 2 (different route/state):
<motion.img layoutId="hero-image" src={img} className="full-screen" />
// Framer Motion animates between them automatically

```

CSS Animations — pure performance:

```

/* Prefer CSS animations for simple, repeating, or performance-critical cases */

/* Entrance animations */
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

@keyframes slideUpFade {
  from { opacity: 0; transform: translateY(16px); }
  to { opacity: 1; transform: translateY(0); }
}

```

```

@keyframes scaleIn {
  from { opacity: 0; transform: scale(0.92); }
  to { opacity: 1; transform: scale(1); }
}

/* Loading states */
@keyframes shimmer {
  0% { background-position: -200% 0; }
  100% { background-position: 200% 0; }
}

.skeleton {
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;
}

@keyframes pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.4; }
}

/* Respect user preferences — always */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}

```

GSAP — for complex, timeline-based animations:

```

import gsap from 'gsap'
import { ScrollTrigger } from 'gsap/ScrollTrigger'
gsap.registerPlugin(ScrollTrigger)

// Timeline — sequence of animations
const tl = gsap.timeline({ delay: 0.5 })
tl.from('.hero-title', { opacity: 0, y: 60, duration: 0.8, ease: 'power3.out' })
  .from('.hero-subtitle', { opacity: 0, y: 40, duration: 0.6, ease: 'power3.out' })
  .from('.hero-cta', { opacity: 0, scale: 0.9, duration: 0.4, ease: 'back.out' },

// ScrollTrigger
gsap.from('.section-content', {
  scrollTrigger: {
    trigger: '.section-content',

```

```
    start: 'top 80%',
    end: 'bottom 20%',
    toggleActions: 'play none none reverse',
  },
  opacity: 0,
  y: 60,
  duration: 0.9,
  stagger: 0.1,
})
```

Part 4 — Backend Engineering

4.1 — API Design Principles

```
// REST API structure — consistent, predictable, versioned
// GET    /api/v1/users          → list users
// POST   /api/v1/users          → create user
// GET    /api/v1/users/:id      → get single user
// PATCH  /api/v1/users/:id      → partial update
// PUT    /api/v1/users/:id      → full replacement
// DELETE /api/v1/users/:id      → delete

// Standard response envelope
interface ApiResponse<T> {
  success: boolean
  data?: T
  error?: {
    code: string
    message: string
    details?: unknown
  }
  meta?: {
    page: number
    limit: number
    total: number
    hasMore: boolean
  }
}

// Error codes — be specific
const ErrorCodes = {
  NOT_FOUND: 'RESOURCE_NOT_FOUND',
  UNAUTHORIZED: 'UNAUTHORIZED',
```

```

    FORBIDDEN: 'FORBIDDEN',
    VALIDATION_ERROR: 'VALIDATION_ERROR',
    DUPLICATE: 'DUPLICATE_RESOURCE',
    RATE_LIMITED: 'RATE_LIMIT_EXCEEDED',
    INTERNAL: 'INTERNAL_SERVER_ERROR',
  } as const

// Hono – the fastest, cleanest API framework
import { Hono } from 'hono'
import { zValidator } from '@hono/zod-validator'
import { z } from 'zod'

const app = new Hono()

const CreateUserSchema = z.object({
  name: z.string().min(1).max(100),
  email: z.string().email(),
  role: z.enum(['user', 'admin']).default('user'),
})

app.post(
  '/users',
  zValidator('json', CreateUserSchema),
  async (c) => {
    const body = c.req.valid('json') // fully typed
    const user = await db.user.create({ data: body })
    return c.json({ success: true, data: user }, 201)
  }
)

```

4.2 — Database Patterns

```

// Prisma schema
// prisma/schema.prisma
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  name        String?
  role        Role       @default(USER)
  posts       Post[]
  profile     Profile?
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt
}

```

```

    @@index([email])
    @@map("users")
  }

// Drizzle (SQL-first, TypeScript-native)
import { pgTable, text, timestamp, uuid } from 'drizzle-orm/pg-core'
import { createInsertSchema, createSelectSchema } from 'drizzle-zod'

export const users = pgTable('users', {
  id: uuid('id').defaultRandom().primaryKey(),
  email: text('email').notNull().unique(),
  name: text('name'),
  createdAt: timestamp('created_at').defaultNow().notNull(),
})

export const InsertUser = createInsertSchema(users)
export const SelectUser = createSelectSchema(users)
export type User = typeof users.$inferSelect
export type NewUser = typeof users.$inferInsert

// Query patterns
// Prisma queries
const users = await prisma.user.findMany({
  where: { role: 'ADMIN', createdAt: { gte: new Date('2024-01-01') } },
  select: { id: true, name: true, email: true },
  orderBy: { createdAt: 'desc' },
  take: 20,
  skip: (page - 1) * 20,
})

// Drizzle queries
const result = await db
  .select({ id: users.id, name: users.name, email: users.email })
  .from(users)
  .where(and(eq(users.role, 'admin'), gte(users.createdAt, new Date('2024-01-01'))
  .orderBy(desc(users.createdAt))
  .limit(20)
  .offset((page - 1) * 20)

```

Part 5 — Web Design Principles in Code

5.1 — Typography

```

/* Typography system */
:root {
  /* Scale: 1.25 ratio (Major Third) */
  --text-xs: 0.640rem; /* 10.24px */
  --text-sm: 0.800rem; /* 12.80px */
  --text-base: 1.000rem; /* 16px — base */
  --text-lg: 1.250rem; /* 20px */
  --text-xl: 1.563rem; /* 25px */
  --text-2xl: 1.953rem; /* 31.25px */
  --text-3xl: 2.441rem; /* 39px */
  --text-4xl: 3.052rem; /* 48.8px */

  /* Line heights */
  --leading-tight: 1.25;
  --leading-snug: 1.375;
  --leading-normal: 1.5;
  --leading-relaxed: 1.625;
  --leading-loose: 2;

  /* Letter spacing */
  --tracking-tight: -0.025em;
  --tracking-normal: 0em;
  --tracking-wide: 0.025em;
  --tracking-wider: 0.05em;
  --tracking-widest: 0.1em;
}

/* Responsive fluid typography */
.hero-heading {
  /* Clamp: min size, preferred size, max size */
  font-size: clamp(2rem, 5vw + 1rem, 4rem);
  line-height: 1.1;
  letter-spacing: -0.02em;
}

/* Optimal reading width */
.prose {
  max-width: 65ch; /* approximately 65 characters */
  line-height: 1.6;
  font-size: 1.125rem;
}

```

5.2 — Spacing & Layout

```

/* Grid system */

```



```

.grid-layout {
  display: grid;
  grid-template-columns: repeat(12, 1fr);
  gap: var(--spacing-4); /* 1rem */
  max-width: 1280px;
  margin: 0 auto;
  padding: 0 var(--spacing-6);
}

/* Responsive without media queries (container queries) */
@container (min-width: 600px) {
  .card-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* CSS logical properties for international support */
.element {
  margin-block: 1rem; /* top and bottom */
  margin-inline: 2rem; /* left and right (flips in RTL) */
  padding-block-start: 1rem; /* top in LTR, top in RTL */
}

/* Modern centering */
.centered {
  display: grid;
  place-items: center;
}
.centered-flex {
  display: flex;
  align-items: center;
  justify-content: center;
}

```

5.3 — Responsive Design

```

// Breakpoints — mobile first always
// xs: 0px      (base — mobile portrait)
// sm: 640px    (mobile landscape / small tablet)
// md: 768px    (tablet portrait)
// lg: 1024px   (tablet landscape / laptop)
// xl: 1280px   (desktop)
// 2xl: 1536px  (large desktop)

// Tailwind responsive classes
<div className="

```

```

flex flex-col          // mobile: stack vertically
md:flex-row           // tablet+: side by side
gap-4
p-4 md:p-8 lg:p-12    // increasing padding at each breakpoint
">
<aside className="w-full md:w-64 lg:w-72 flex-shrink-0">
  {/* Sidebar */}
</aside>
<main className="flex-1 min-w-0"> {/* min-w-0 prevents flex overflow */}
  {/* Content */}
</main>
</div>

// Container queries (more precise than viewport queries)
<div className="@container">
  <div className="grid @sm:grid-cols-2 @lg:grid-cols-3 gap-4">
    {/* Responds to container width, not viewport */}
  </div>
</div>

```

Part 6 — Performance Engineering

6.1 — Core Web Vitals

```

// LCP (Largest Contentful Paint) — target < 2.5s
// - Preload hero images
<link rel="preload" href="/hero.jpg" as="image" />
// - Use next/image with priority for above-fold images
<Image src="/hero.jpg" priority sizes="100vw" />

// FID/INP (Interaction to Next Paint) — target < 200ms
// - Avoid long tasks (>50ms) on the main thread
// - Use Web Workers for heavy computation
// - Debounce expensive event handlers

// CLS (Cumulative Layout Shift) — target < 0.1
// - Always set width and height on images
// - Reserve space for dynamic content (skeleton loaders)
// - Avoid inserting content above existing content

// Bundle size analysis
// Next.js: npx @next/bundle-analyzer
// Vite: npx vite-bundle-visualizer

```

```
// Webpack: webpack-bundle-analyzer

// Code splitting strategies
const HeavyChart = dynamic(() => import('./Chart'), {
  loading: () => <Skeleton className="h-64" />,
  ssr: false // if browser-only
})

// Image optimization
<Image
  src="/photo.jpg"
  alt="Description"
  width={800}
  height={600}
  sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 33vw"
  placeholder="blur"
  blurDataURL="data:image/jpeg;base64,..."
/>
```

6.2 — Caching Strategy

```
// React Query caching
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 5 * 60 * 1000, // 5 min — data is fresh
      gcTime: 10 * 60 * 1000, // 10 min — keep in memory
      retry: 3,
      refetchOnWindowFocus: true,
    }
  }
})

// HTTP cache headers
// In Next.js Route Handler:
return NextResponse.json(data, {
  headers: {
    'Cache-Control': 'public, max-age=60, stale-while-revalidate=30',
    'CDN-Cache-Control': 'public, max-age=3600',
  }
})

// Optimistic updates — feel instant
const { mutate } = useMutation({
  mutationFn: updateUser,
  onMutate: async (newData) => {
```

```

    await queryClient.cancelQueries({ queryKey: ['user', id] })
    const previous = queryClient.getQueryData(['user', id])
    queryClient.setQueryData(['user', id], newData) // update immediately
    return { previous }
  },
  onError: (err, newData, context) => {
    queryClient.setQueryData(['user', id], context?.previous) // rollback
  },
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: ['user', id] })
  }
})

```

Part 7 — Accessibility (Non-Negotiable)

```

// Semantic HTML — the foundation
// Never: <div onClick={...}>Click me</div>
// Always: <button onClick={...}>Click me</button>

// ARIA — only when semantic HTML isn't enough
<button
  aria-label="Close modal"           // when no visible text
  aria-expanded={isOpen}            // for toggles
  aria-controls="modal-id"          // links trigger to target
  aria-describedby="tooltip-id"     // links element to description
  aria-disabled={isLoading}         // non-focusable disabled
>
  <CloseIcon aria-hidden="true" />   // decorative icons hidden from SR
</button>

// Focus management
// Trap focus in modals
// Return focus to trigger after modal closes
// Skip navigation links for keyboard users

// Color contrast
// Normal text: 4.5:1 minimum (WCAG AA)
// Large text (18px+): 3:1 minimum
// UI components and focus indicators: 3:1 minimum

// Screen reader announcements for dynamic content
const [announcement, setAnnouncement] = useState('')
// Trigger: setAnnouncement('Form submitted successfully')
<div role="status" aria-live="polite" className="sr-only">

```

```

    {announcement}
  </div>

  // Focus-visible vs focus
  // Show focus styles for keyboard, hide for mouse
  <button className="
    focus:outline-none
    focus-visible:ring-2
    focus-visible:ring-primary
    focus-visible:ring-offset-2
  ">

  // Always respect reduced motion
  const prefersReducedMotion = window.matchMedia('(prefers-reduced-motion: reduce)'
```

Part 8 — Testing

```

// Unit tests - Vitest (fast, Vite-native)
import { describe, it, expect, vi } from 'vitest'

describe('formatCurrency', () => {
  it('formats USD correctly', () => {
    expect(formatCurrency(1234.56, 'USD')).toBe('$1,234.56')
  })
  it('handles zero', () => {
    expect(formatCurrency(0, 'USD')).toBe('$0.00')
  })
  it('handles negative numbers', () => {
    expect(formatCurrency(-100, 'USD')).toBe('-$100.00')
  })
})

// Component tests - React Testing Library
import { render, screen, fireEvent, waitFor } from '@testing-library/react'
import userEvent from '@testing-library/user-event'

describe('Button', () => {
  it('calls onClick when clicked', async () => {
    const user = userEvent.setup()
    const onClick = vi.fn()
    render(<Button onClick={onClick}>Click me</Button>)
    await user.click(screen.getByRole('button', { name: 'Click me' }))
    expect(onClick).toHaveBeenCalledTimes(1)
  })
})
```

```

    it('shows loading state', () => {
      render(<Button isLoading>Submit</Button>)
      expect(screen.getByRole('button')).toBeDisabled()
    })
  })

// E2E tests - Playwright
import { test, expect } from '@playwright/test'

test('user can complete checkout', async ({ page }) => {
  await page.goto('/products')
  await page.click('[data-testid="add-to-cart-1"]')
  await page.click('[data-testid="cart-icon"]')
  await expect(page.locator('[data-testid="cart-count"]')).toHaveText('1')
  await page.click('[data-testid="checkout-btn"]')
  await page.fill('[name="email"]', 'test@example.com')
  await page.click('[type="submit"]')
  await expect(page).toHaveURL('/checkout/success')
})

// API tests
test('POST /api/users creates a user', async ({ request }) => {
  const response = await request.post('/api/users', {
    data: { name: 'Test User', email: 'test@example.com' }
  })
  expect(response.status()).toBe(201)
  const body = await response.json()
  expect(body.data).toMatchObject({ name: 'Test User', email: 'test@example.com' })
  expect(body.data.id).toBeDefined()
})

```

Part 9 — Developer Experience

9.1 — Project Setup Standards

```

// package.json scripts
{
  "scripts": {
    "dev": "next dev --turbo",
    "build": "next build",
    "start": "next start",
    "lint": "eslint . --ext .ts,.tsx --fix",

```

```

    "typecheck": "tsc --noEmit",
    "test": "vitest",
    "test:e2e": "playwright test",
    "test:ui": "vitest --ui",
    "format": "prettier --write .",
    "prepare": "husky install",
    "analyze": "ANALYZE=true next build"
  }
}

// tsconfig.json – strict mode always
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "esnext"],
    "strict": true, // enables all strict checks
    "noUncheckedIndexedAccess": true, // array access returns T | undefined
    "exactOptionalPropertyTypes": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "moduleResolution": "bundler",
    "paths": {
      "@/*": ["../src/*"] // path aliases
    }
  }
}

// .eslintrc.js
module.exports = {
  extends: [
    'next/core-web-vitals',
    'plugin:@typescript-eslint/recommended-type-checked',
    'plugin:unicorn/recommended',
    'prettier',
  ],
  rules: {
    '@typescript-eslint/no-explicit-any': 'error',
    '@typescript-eslint/consistent-type-imports': 'error',
    'no-console': ['warn', { allow: ['error', 'warn'] }],
  }
}

```

9.2 — Git Workflow

```
# Commit message convention (Conventional Commits)
feat: add user authentication
fix: resolve cart total calculation bug
chore: update dependencies
docs: add API documentation
refactor: extract payment logic to service
perf: optimize image loading
test: add checkout flow E2E tests
ci: add GitHub Actions workflow

# Branch naming
feature/user-authentication
fix/cart-total-bug
chore/dependency-updates

# .husky/pre-commit
#!/usr/bin/env sh
npx lint-staged # runs linting and formatting on staged files

# lint-staged config in package.json
{
  "lint-staged": {
    "*.ts,tsx)": ["eslint --fix", "prettier --write"],
    "*.json,md,css)": ["prettier --write"]
  }
}
```

Quality Checklist

Before shipping any code:

CODE QUALITY

- [] TypeScript strict mode – zero type errors
- [] No console.log left in production code
- [] All async functions have error handling (try/catch or .catch())
- [] No hardcoded secrets or API keys
- [] Environment variables documented in .env.example
- [] All props typed with interfaces or types
- [] No any types without justification

UI/UX QUALITY

- [] Responsive at 375px, 768px, 1280px minimum
- [] Loading states on all async operations

- [] Empty states on all data lists
- [] Error states on all fallible operations
- [] Forms have validation messages
- [] All interactive elements have hover and focus states
- [] Color contrast meets WCAG AA (4.5:1 for text)
- [] Animations respect prefers-reduced-motion
- [] All images have alt text
- [] Buttons have accessible labels

PERFORMANCE

- [] No unnecessary re-renders (check with React DevTools)
- [] Images optimized and properly sized
- [] No blocking scripts in <head>
- [] Code split at route level minimum
- [] API responses cached appropriately

PRODUCTION READINESS

- [] Error boundaries around major UI sections
- [] Logging for critical operations
- [] Input validation on both client and server
- [] CORS configured correctly
- [] Rate limiting on public API routes
- [] Environment-specific configuration verified
- [] Build succeeds with zero warnings